Headless Execution and TargetLocator Interface
WebDriver - Lecture - 8

Agenda

- Point Class in Selenium Webdriver
- DesiredCapabilities Class
- FirefoxOptions And ChromeOptions
- Headless Execution
- HtmlUnitDriver
- TargetLocator Interface
- Switching Window
- Get All Window Handles
- Quiz
- Homework

Point Class in Selenium Webdriver

- **Point class** is used to get the web elements ”***x and y”*** coordinates on web page.

- ”***x and y”*** coordinates of a web element is measured in **“x” and “y” pixels**.
  - ***x* pixels** means the **horizontal position** of an element on a **page from the left side**
  - ***y* pixels** means the **vertical position** of an element on a **page from the top**

- **Point class** is provided in “**org.openqa.selenium**” package.

- WebElement’s **getLocation()** method returns **Point class** object.

- **Difference** of **“x” and “y” coordinates** of destination element with **“x” and “y” coordinates** of source element gives “**x-offset**” and “**y-offset**” respectively.

```
Point currentLocation = source.getLocation();
Point finalLocation = destination.getLocation();

int xOffset = finalLocation.getX() - currentLocation.getX()+1;
int yOffset = finalLocation.getY() - currentLocation.getY()+1;

act.dragAndDropBy(source, xOffset, yOffset).build().perform();
```

DesiredCapabilities Class

- **DesiredCapabilities** class in **WebDriver API** is a series of key/value pairs that stores the browser properties like **browser name**, **browser version**, the **path of the browser driver** in the system, **supports javascript or not** etc. to determine the behaviour of the browser at a run time.
- This class is provided in **"org.openqa.selenium.remote" package.**
- Mostly **DesiredCapabilities** class is used with **Selenium Grid** using "**RemoteWebDriver**" class to execute multiple testcases on different browsers with different version and different Operating systems.
- **DesiredCapabilities** class inherits from **MutableCapabilities** class which further inherits from **Capabilities** interface**.**
- **DesiredCapabilities** class is used to set some **specific testing environment** like web browser, Mobile device, mobile emulator or mobile simulator using **setCapability()** method**.**

```
DesiredCapabilities capabilities = new DesiredCapabilities();
capabilities.setBrowserName("firefox");
capabilities.setCapability("platform",Platform.WINDOWS);
capabilities.setCapability(ForSeleniumServer.PROXYING_EVERYTHING, true);
capabilities.setCapability(CapabilityType.ACCEPT_SSL_CERTS, true);
capabilities.setCapability(CapabilityType.SUPPORTS_ALERTS, true);
capabilities.setCapability(CapabilityType.SUPPORTS_JAVASCRIPT, true);
capabilities.setCapability("marionette", true);
```

FirefoxOptions And ChromeOptions

- Both of these classes are used to handle properties of **Firefox** and **Chrome** browsers via **geckodriver** and **chromedriver** respectively.
- Both are the **child class** of **MutableCapabilities** class which is also parent class of **DesiredCapabilities class**.
- Object of both of these classes are passed an **argument** to **FirefoxDriver** and **ChromeDriver** classes.
- Using **Browser Options** class, we can set options like making the **browser headless**, **load extensions**, **set binaries, set proxy** and so on.
- **Chrome Incognito mode**: We need to add "**--incognito**" argument with **addArguments(String args)** method to start chrome in **incognito mode**.

```
FirefoxOptions ffOptions = new FirefoxOptions();
ffOptions.addPreference("browser.startup.page", 1);
ffOptions.addPreference("browser.startup.homepage", "https://app.box.com");
ffOptions.setCapability("marionette", true);
ffOptions.setCapability(ForSeleniumServer.PROXYING_EVERYTHING, true);
ffOptions.setCapability(CapabilityType.ACCEPT_SSL_CERTS, true);
ffOptions.setCapability(CapabilityType.SUPPORTS_ALERTS, true);
ffOptions.setCapability(CapabilityType.SUPPORTS_JAVASCRIPT, true);
ffOptions.setCapability("platform",Platform.WINDOWS);
driver = new FirefoxDriver(ffOptions);
```

```
ChromeOptions options = new ChromeOptions();
options.addArguments("--incognito");
options.addArguments("start-maximized");
ChromeDriver driver = new ChromeDriver(options);
```

Headless Execution

- A **headless execution** is a browser simulation that **does not** have a **user interface** or GUI.

- **Headless execution** is useful when running tests especially in a **CI environment** where nobody is watching the visuals and there is **no need** of browser GUI.

- **Chrome Headless** is achieved by adding two options through **ChromeOptions**: **headless** and the **window-size**.
  - The **headless option** will tell **Google Chrome** to execute in headless mode.
  - The **window-size** is a way to control the **responsiveness**

- **Firefox Headless** is achieved by setting **option headless** to ***"true"*** through **FirefoxOptions.**

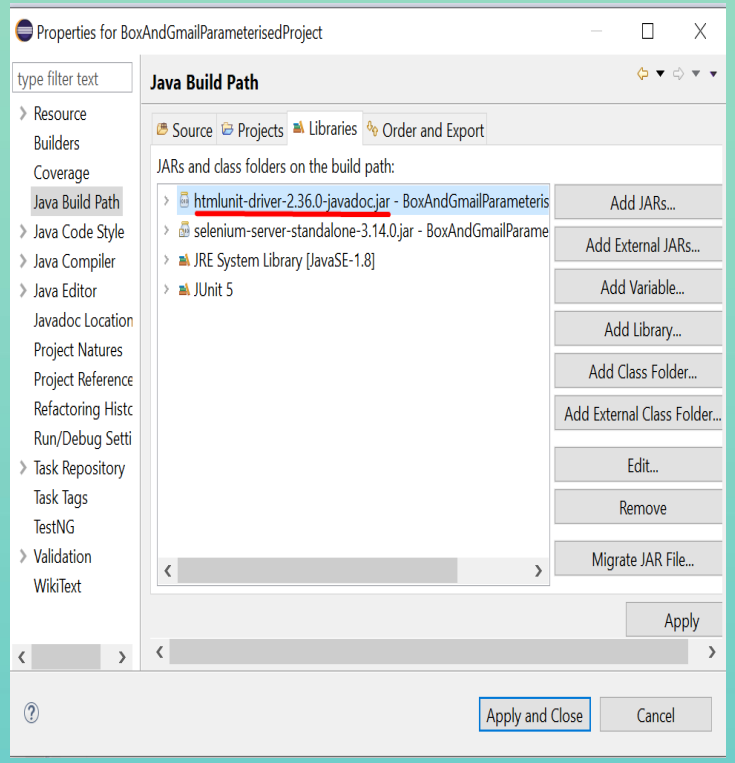
- **Taking screenshot** is possible while working with both **Firefox Headless** and **Chrome Headless**.

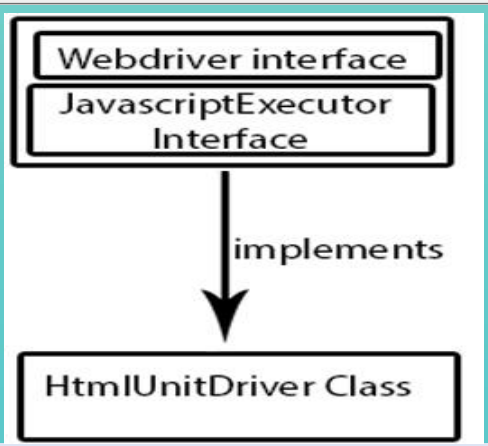
```
ChromeOptions options = new ChromeOptions();
options.addArguments("headless");
options.addArguments("window-size=1200x600");

WebDriver driver = new ChromeDriver(options);
```

```
FirefoxOptions options = new FirefoxOptions();
options.setHeadless(true);

WebDriver driver = new FirefoxDriver(options);
```

HtmlUnitDriver

- **HtmlUnit** is a **java based** implementation of a web browser **without a user interface**.
- This class is in "**org.openqa.selenium.htmlunit** " package.
- To use **HtmlUnitDriver class**, include **htmlunit-driver-xxx.jar** as a **dependency** in your project's build-path.
- **HtmlUnitDriver** class is direct implementation of **WebDriver** and implements **JavascriptExecutor**.
- This class is not in hierarchy of **RemoteWebDriver** class and does not implement **TakesScreenshot** interface.
- **Constructors** for doing headless execution with **HtmlUnitDriver**
  - **HtmlUnitDriver()** - Constructs a new instance with **JavaScript** disabled, and the default BrowserVersion
  - **HtmlUnitDriver(boolean enableJavascript)** - Constructs a new instance, specify **JavaScript** support and using the default BrowserVersion.
  - **HtmlUnitDriver(com.gargoylesoftware.htmlunit.BrowserVersion version)** - Constructs a new instance with the specified **BrowserVersion**.
  - **HtmlUnitDriver(com.gargoylesoftware.htmlunit.BrowserVersion version, boolean enableJavascript)** - Constructs a new instance with the specified **BrowserVersion** and the **JavaScript** support.





TargetLocator Interface

- **TargetLocator** is an inner interface of **WebDriver** for changing the focus of the driver to another **frame, window** or **alert**.
- **TargetLocator** should never be instantiated directly, rather obtain its instance with **switchTo()** method of WebDriver.
  - **TargetLocator switchTo()** – This method when used with **WebDriver** instance, returns the object of **TargetLocator** interface.

```
//*****************************
TargetLocator tl = driver.switchTo();
//*****************************
```

Instance methods of TargetLocator Interface

- **WebDriver window(String nameOrHandle)** - **Switch** the focus of future commands for this driver to the **window** with the **given name/handle**.
- **WebDriver frame(int index) -** Select a frame by its **(zero-based) index** and **switches** to it.
- **WebDriver frame(String nameOrId)** - Select a frame by its **name or ID** and **switches** to it.
- **WebDriver frame(WebElement frameElement)** - Select a **frame** by **frame(webelement)** and **switches** to it.
- **Alert alert()** – **Switches** to the **alert** dialog.

Switching Window

- **TargetLocator** has **window()** method which **switches** the focus of future commands of **WebDriver** to the **new tab or window**
  - **window()** method returns **WebDriver instance** of new window.
  - **window()** method takes **one parameter** which is **handle/id** of new window.
  - A **window handle** is a **alphanumeric ID** that gets assigned to each window when **object** of **WebDriver** is instantiated.
  - We need this **ID** to identify each window to **switch** between them.

- **WebDriver** interface has method **getWindowHandle()** which is used with **WebDriver** object and returns the window handle of current **focused window** as a **String** representation.

String getWindowHandle() – Returns **handle(String)** to this window that uniquely identifies it within this driver instance

```
//gets the window handle of current focused window.
String homePage = driver.getWindowHandle();
driver.switchTo().window(homePage);
```

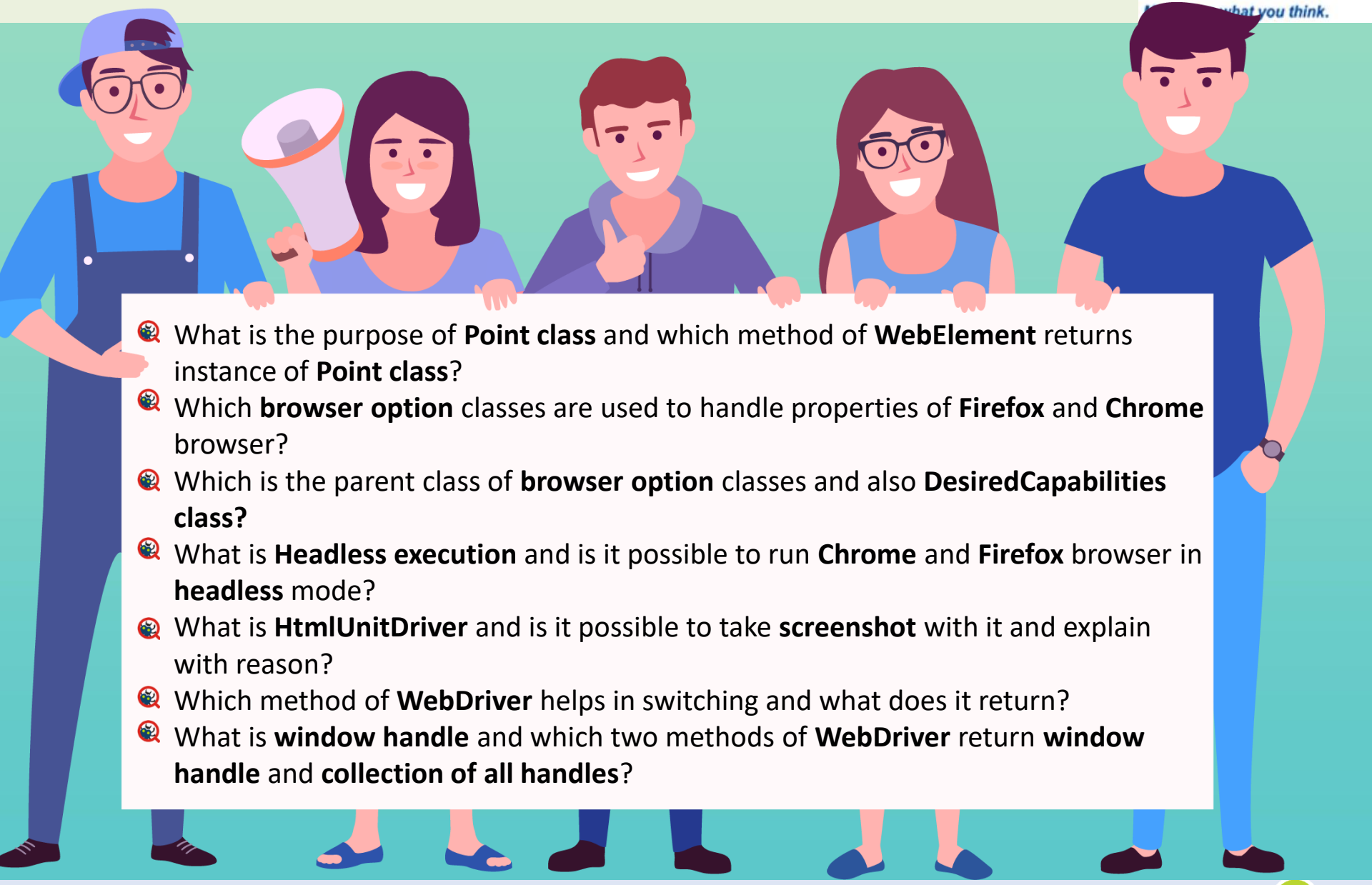
Get All Window Handles

- **WebDriver** interface has method **getWindowHandles()** which is used with **WebDriver** object and returns the "**java.util.Set**" collection for **all opened WebDriver windows as String**.
- For working on **new tab or window** like getting new window title or clicking on new window element, we first need to **switch** to it.

  Set<String> getWindowHandles() –
  - Return a **set of window handles** which can be used to **iterate** over all **open windows** for this **WebDriver instance**
  - So, we get **collection** of **String** for all **unique window handles.**

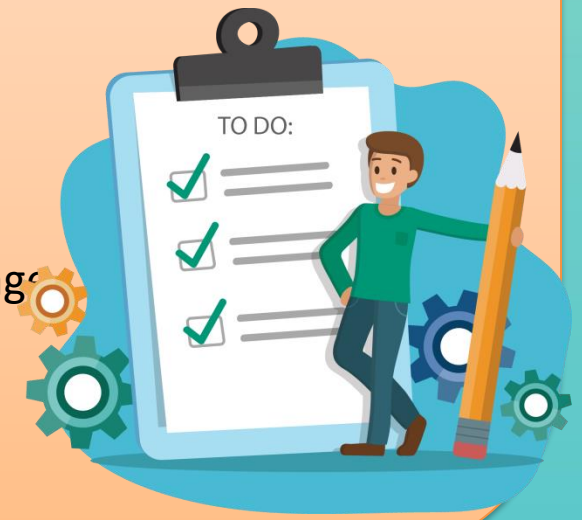
```
 TargetLocator tl = driver.switchTo();
String homePage = driver.getWindowHandle();
//*********************************************
//get all window handles in a collection
java.util.Set<String> windows = driver.getWindowHandles();
//*********************************************

for (String windowHandle : windows) {
    if (!windowHandle.equals(homePage)) {
        WebDriver dr = tl.window(windowHandle);
        dr.close();
        driver.switchTo().window(homePage);

    }
}
```

Quiz



- What is the purpose of **Point class** and which method of **WebElement** returns instance of **Point class**?
- Which **browser option** classes are used to handle properties of **Firefox** and **Chrome** browser?
- Which is the parent class of **browser option** classes and also **DesiredCapabilities class?**
- What is **Headless execution** and is it possible to run **Chrome** and **Firefox** browser in **headless** mode?
- What is **HtmlUnitDriver** and is it possible to take **screenshot** with it and explain with reason?
- Which method of **WebDriver** helps in switching and what does it return?
- What is **window handle** and which two methods of **WebDriver** return **window handle** and **collection of all handles**?

Homework

- Write a **WebDriver** script and
  - Navigate to http://qacult.com/selenium-webdriver-practice
  - Get the handle of current window
  - Scroll and click on “Click to open a small window!”
  - Click on “Click to open a new browser window!”
  - Get all opened current window handles.
  - Print out the size of opened windows
  - Get the title of each new window and close them

- Write a **WebDriver** script in box portal
  - Navigate to https://app.box.com
  - Login to the app
  - Verify the title of “All Files” page
  - Click on Notes button in left panel of “All Files” page
  - Verify new tab is opened
  - Verify title of new tab is "Box Notes“
  - Close the new tab